HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY, VNU-HCM
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

Semester 261

Assignment 1 Report

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models:
A Comparative Study for Image Classification

Group Placeholder

Instructor	Le Thanh Sach
Students	**Student ID**
Tran Lam Anh	2352067
Vu Duc Viet Anh	2352074
Tran Nguyen Giap	2352284
Le Nguyen Khang	2352470

Ho Chi Minh City, 28 October 2026

Abstract

We compare the accuracy and classification errors of Linear, MLP, CNN, GRU, LSTM, and Transformer models on Fashion-MNIST. Each model trains for 50 epochs with seed 42 on the same data split. We evaluate the checkpoint with the lowest validation loss on the official 10,000-image test set. CNN achieves the highest accuracy at 92.59% and a macro-F1 of 0.9261. It exceeds Transformer accuracy by 2.16 percentage points while using 421,738 parameters compared with 1,191,050. Confusions among upper-body garments account for 68.4% of CNN errors. CNN correctly identifies 79.5% of Shirt images, compared with 62.1% for MLP. We also compare how the models classify the same images. Although CNN has higher overall accuracy, MLP correctly classifies 234 images that CNN misses. Each model uses one seed, so we cannot assess variation across training runs. The runs also use different timing conditions, which prevents a speed comparison.

Table of Contents

Abstract **i**

1 Problem and data description **1**

1.1 Task and research question 1

1.2 Dataset and study scope 1

1.3 Exploratory data analysis 2

1.4 Splits and leakage prevention 4

1.5 Preprocessing and augmentation 4

2 Methodology **5**

2.1 Training and evaluation pipeline 5

2.2 Model architectures 5

2.3 Architecture comparison 8

2.4 Loss and evaluation metrics 9

2.5 Training configuration 10

2.6 Comparison protocol 10

3 Implementation results **11**

3.1 Test performance and model size 11

3.2 Learning behavior 12

3.3 Per-class performance and error analysis 12

3.4 Predictions on shared images 14

3.5 Limitations 15

3.6 Discussion and conclusion 15

AI usage disclosure **16**

References **16**

1 Problem and data description

1.1 Task and research question

We classify each image into one of ten clothing categories. This task can help organize online product catalogs. We compare the models on the same data splits to identify differences in accuracy and common errors.

For Fashion-MNIST, the model receives a normalized grayscale tensor $x \in \mathbb{R}^{1\times28\times28}$ and returns ten class scores $f_\theta(x) \in \mathbb{R}^{10}$. Its prediction is $\hat{y} = \arg\max_{k\in\{0,\dots,9\}} f_\theta(x)_k$. Low resolution and similar garment outlines make some categories difficult to separate.

1.2 Dataset and study scope

Fashion-MNIST contains 70,000 Zalando product images with supplied category labels: 60,000 official training images and 10,000 official test images [3]. Figure 1 shows the ten categories.

We use the authors' MIT-licensed dataset in compressed IDX format (30.88 MB) [4]. These results apply to small product images from ten known categories. We do not evaluate full scenes or unseen categories.

Fashion-MNIST is the main comparison dataset. MNIST handwritten digits are used for development, and CIFAR-10 colour images provide a separate extension [5, 6]. Table 1 lists the dataset splits.

Table 1: Dataset roles, native image sizes, and split counts. Image dimensions are height × width × channels.

Dataset	Role	Image size	Train	Validation	Test
Fashion-MNIST	Main comparison	$28 \times 28 \times 1$	54,000	6,000	10,000
MNIST	Development	$28 \times 28 \times 1$	54,000	6,000	10,000
CIFAR-10	Extension	$32 \times 32 \times 3$	45,000	5,000	10,000

1.3 Exploratory data analysis

We analyze the 60,000 original Fashion-MNIST training images. Each image is 28×28 pixels, and each class has 6,000 images, so the dataset is balanced. The official test set has 1,000 images per class.

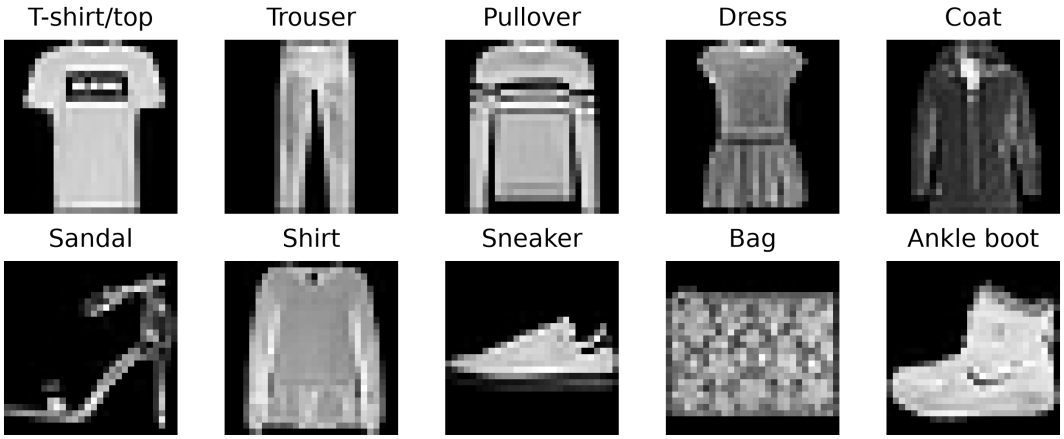


Figure 1: The first training image from each Fashion-MNIST class.

Figure 2 shows the average image for each class. Pullover, Coat, and Shirt share a broad torso and sleeve outline, while Trouser and the footwear classes have more distinct shapes. Averaging keeps features that recur in similar positions but blurs details that vary between products. The shared sleeve and torso shapes are therefore easier to see here than in a single example, while individual collars or textures are less visible.

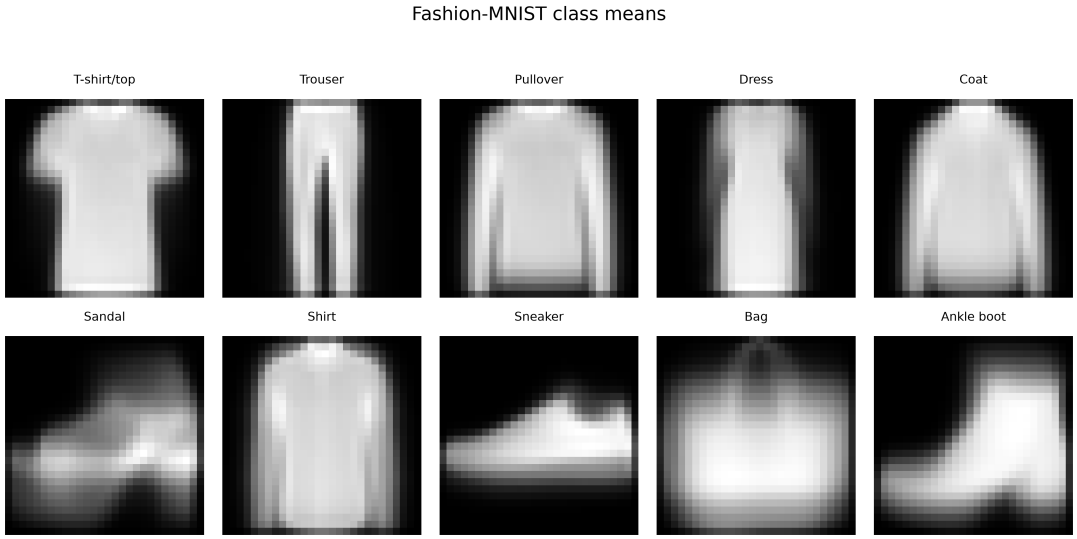


Figure 2: Average image for each Fashion-MNIST class, using all 6,000 official training images per class.

About 53.0% of pixels have intensities below 8 because much of each image is dark background. Figure 3 shows how much of each image the product occupies, counting pixels with intensity above 32 as foreground. Average coverage ranges from 24.5% for Sandal to 59.8% for Pullover. Shirt and Coat have similar average coverage, about 56%.

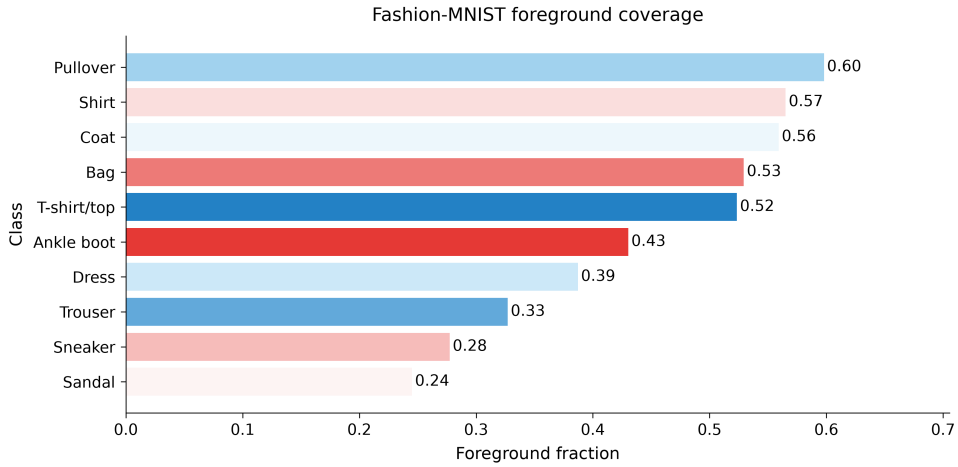


Figure 3: Average foreground fraction by Fashion-MNIST class, using the official training set and an intensity threshold of 32 on the 0–255 scale.

The heatmap in Figure 4 puts these shape similarities into numbers. Pullover and Coat have the highest cosine similarity at 0.86, while Sandal and Sneaker reach 0.82. The closest pairs therefore extend beyond upper-body garments. This comparison uses centred class averages, so it describes typical shapes rather than variation within each class. Subtracting the mean image across classes reduces the common background and overall product layout, making the comparison focus on how each class differs from that shared pattern.

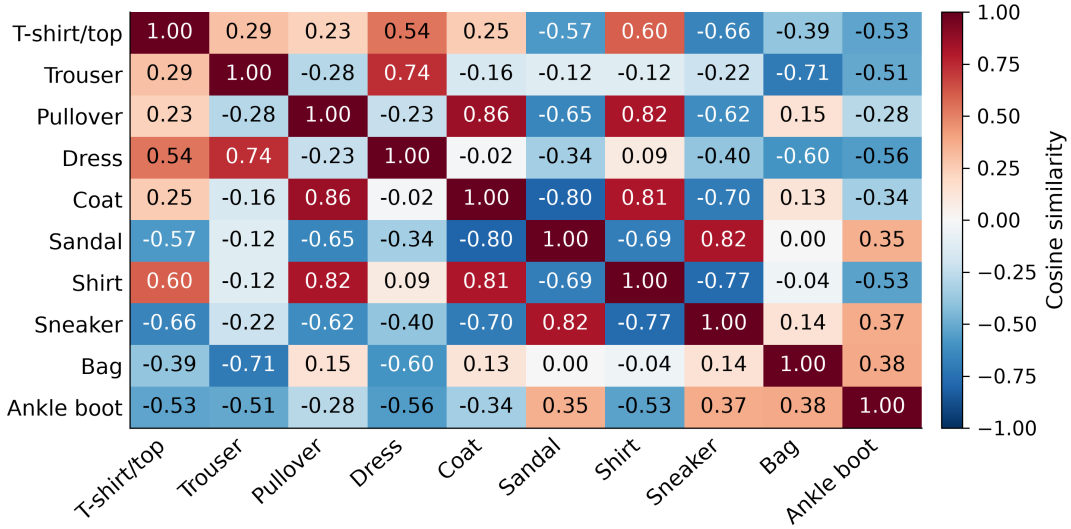


Figure 4: Cosine similarity between centred Fashion-MNIST class-average images, computed from the 60,000 official training images.

1.4 Splits and leakage prevention

For each dataset, a random permutation with seed 42 reserves 10% of the official training set for validation. The split unit is one image, and sampling is not stratified. For Fashion-MNIST, class counts therefore range from 5,353 to 5,445 in training and from 555 to 647 in validation. All models use the same indices for a given dataset. Using identical splits ensures that models do not receive different training or validation images. The small differences in class counts come from random sampling rather than class-based selection.

We use validation loss to select checkpoints, while the official test sets are reserved for final evaluation. Although training and validation indices do not overlap, we have not checked for duplicate images or images of the same product across splits.

1.5 Preprocessing and augmentation

MNIST and Fashion-MNIST keep their native resolution. Pixels are converted to float32 in $[0, 1]$, then normalized as $x' = (x - 0.5)/0.5$. Training adds random rotation within $\pm 10^\circ$ and translation up to 10% of image size. These transformations vary the position and orientation of a product without changing its target label, reducing reliance on its exact pixel location. Normalization maps the original intensity range to $[-1, 1]$.

CIFAR-10 uses four-pixel padding, a 32×32 random crop, and a random horizontal flip for training. We normalize the red, green, and blue channels using the fixed means $(0.4914, 0.4822, 0.4465)$ and standard deviations $(0.2470, 0.2435, 0.2616)$. These are channel statistics of the original 50,000-image CIFAR-10 training set after scaling pixels to $[0, 1]$. For each channel, we subtract its mean and divide by its standard deviation to centre and rescale the input. Validation and test images use the same constants without random augmentation.

For example, a Fashion-MNIST validation batch has 128 grayscale images of 28×28 pixels, with one class label per image.

2 Methodology

2.1 Training and evaluation pipeline

We train all classifiers from scratch using the shared data splits in Section 1.4. Each training batch passes through the model to produce ten class scores, and we update the weights to minimize cross-entropy. After each epoch, we evaluate validation loss with dropout disabled and batch normalization using its running statistics. We use the checkpoint with the lowest validation loss for testing. This allows the selected model to come from an earlier epoch if validation performance deteriorates before training ends. Every model still receives the same 50-epoch training budget.

2.2 Model architectures

Each model maps an image to a feature vector $h \in \mathbb{R}^d$ and then computes ten logits, $z = Wh + b$. The following descriptions use Fashion-MNIST shapes. MNIST uses the same dimensions. CIFAR-10 has three-channel 32×32 images. Table 2 summarizes feature dimensions and parameter counts.

2.2.1 Linear classifier

The linear model flattens an image to $x \in \mathbb{R}^{784}$ and computes $z = Wx + b$, where $W \in \mathbb{R}^{10\times784}$ and $b \in \mathbb{R}^{10}$. Each class score is an affine function of the pixels. Flattening keeps the pixel order. The model has no convolution or shared spatial filters and is our baseline. It tests how far class-specific weighted sums of pixels can go before adding nonlinear layers or explicitly modelling spatial relationships.

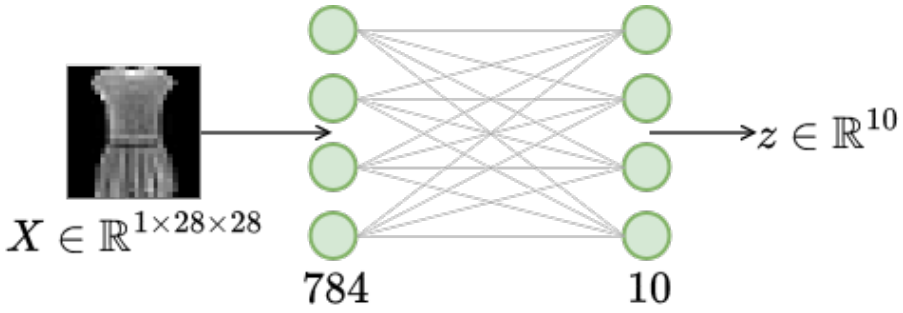


Figure 5: Linear classifier. The flattened image is mapped to ten class logits.

2.2.2 Multilayer perceptron

The MLP applies two hidden affine layers of widths 256 and 128 to the flattened image. Each hidden layer is followed by ReLU and dropout with probability 0.2. A linear head maps the 128-dimensional features to ten logits. The MLP learns nonlinear combinations of pixels but does not share weights across image positions. Comparing it with Linear tests whether nonlinear pixel combinations improve classification, while its comparison with CNN also introduces local connectivity and shared filters.

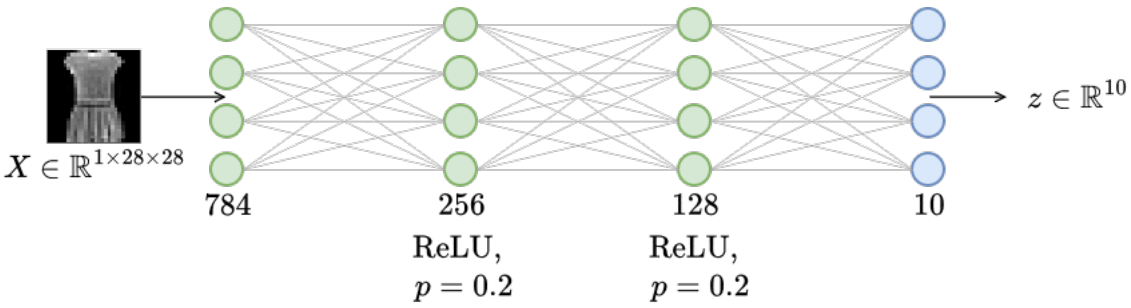


Figure 6: MLP with hidden widths 256 and 128, ReLU, and dropout.

2.2.3 Convolutional neural network

The CNN applies two convolutional blocks with 32 and 64 channels. Each block contains a 3×3 convolution with stride 1 and padding 1, batch normalization, ReLU, and 2×2 max pooling with stride 2. Convolution biases are disabled. For a 28×28 image, the blocks produce $32 \times 14 \times 14$ and $64 \times 7 \times 7$ feature maps. Flattening gives 3,136 values, followed by a 128-unit linear layer and ReLU. A ten-class linear head produces the logits. For CIFAR-10, the final feature map is $64 \times 8 \times 8$, so the hidden layer receives 4,096 values.

Convolution shares local filters across image positions. Flattening preserves the positions of their responses, and the hidden layer can combine responses across the full feature map. The classifier has 421,738 parameters for grayscale images and 545,194 for CIFAR-10.

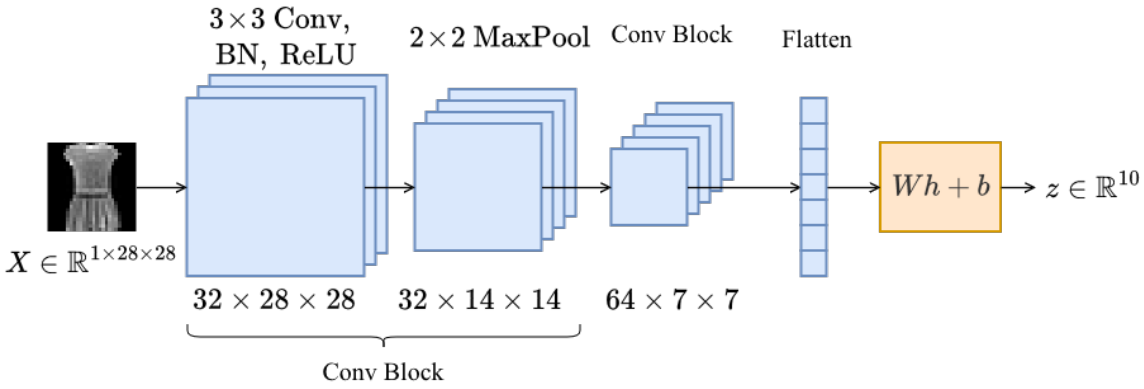


Figure 7: CNN for MNIST and Fashion-MNIST. Flattened convolutional features feed a 128-unit hidden layer and the classification head.

2.2.4 Gated recurrent unit

The GRU reads the image rows from top to bottom. An input of shape $C \times H \times W$ becomes a sequence of H vectors in $\mathbb{R}^{CW}$. Fashion-MNIST gives 28 steps with 28 values per step. CIFAR-10 gives 32 steps with 96 values per step.

The model has one recurrent layer with hidden size 128 and initial state $h_0 = 0$. The updates

follow the GRU equations in PyTorch documentation [8],

$$
\begin{aligned}
r_t &= \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}),\\
u_t &= \sigma(W_{iu}x_t + b_{iu} + W_{hu}h_{t-1} + b_{hu}),\\
\widetilde{h}_t &= \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn}))\,,\\
h_t &= (1 - u_t) \odot \widetilde{h}_t + u_t \odot h_{t-1}.
\end{aligned}
$$

Here, σ is the sigmoid function and $\odot$ denotes elementwise multiplication. The reset gate controls how the previous state contributes to the candidate state. The update gate controls how much of the previous state is kept. All steps share the same parameters. The final state h_H feeds a linear classifier. We do not use recurrent dropout. Reading rows in sequence lets the model combine information from the top and bottom of a product, but all rows must ultimately be summarized in the final 128-dimensional state.

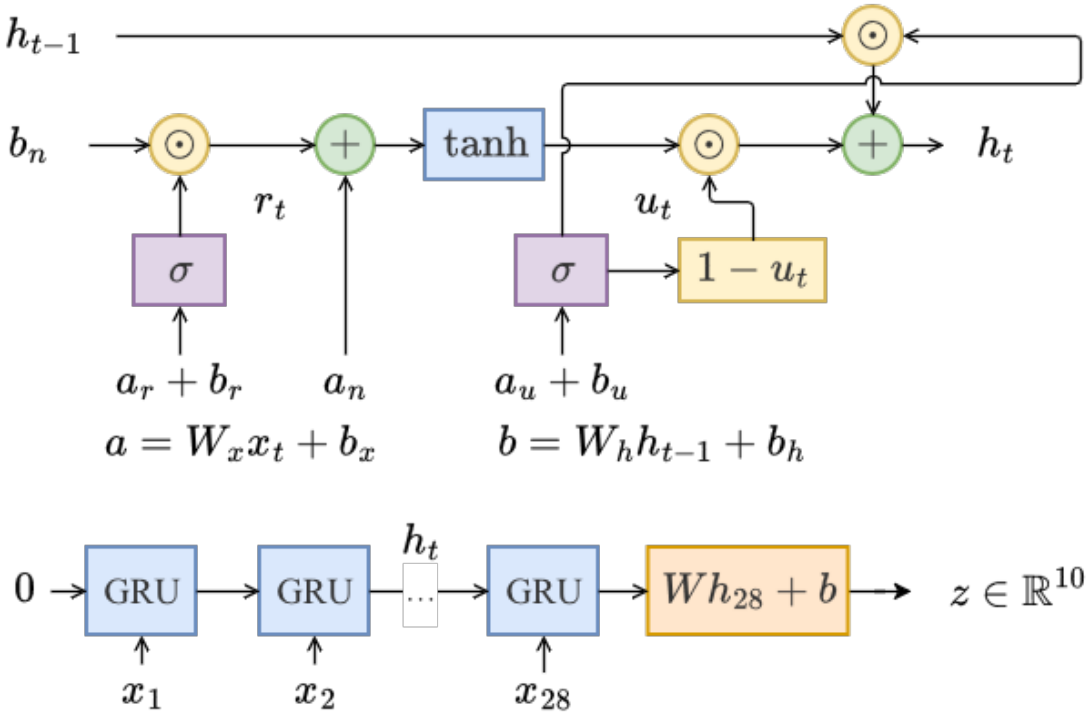


Figure 8: GRU cell and row-sequence classifier. The final hidden state feeds a linear classifier.

2.2.5 Long short-term memory

The LSTM has one layer with hidden size 128. It reads the same row sequence as the GRU and applies a linear head to the final hidden state. It also maintains a cell state c_t. Both states start at zero, and the updates follow the LSTM equations in PyTorch documentation [9],

$$
\begin{aligned}
q_t &= W_x x_t + b_x + W_h h_{t-1} + b_h,\\
(i_t, f_t, g_t, o_t) &= \big(\sigma(q_{t,i}), \sigma(q_{t,f}), \tanh(q_{t,g}), \sigma(q_{t,o})\big),\\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t,\\
h_t &= o_t \odot \tanh(c_t),
\end{aligned}
$$

where $q_t \in \mathbb{R}^{512}$ is divided into four 128-dimensional vectors. The forget gate scales the existing cell state, the input gate controls the candidate information added to it, and the output gate

controls which part of the updated cell state reaches the hidden state. This separates stored memory from the state passed to the classifier, whereas the GRU uses one state for both.

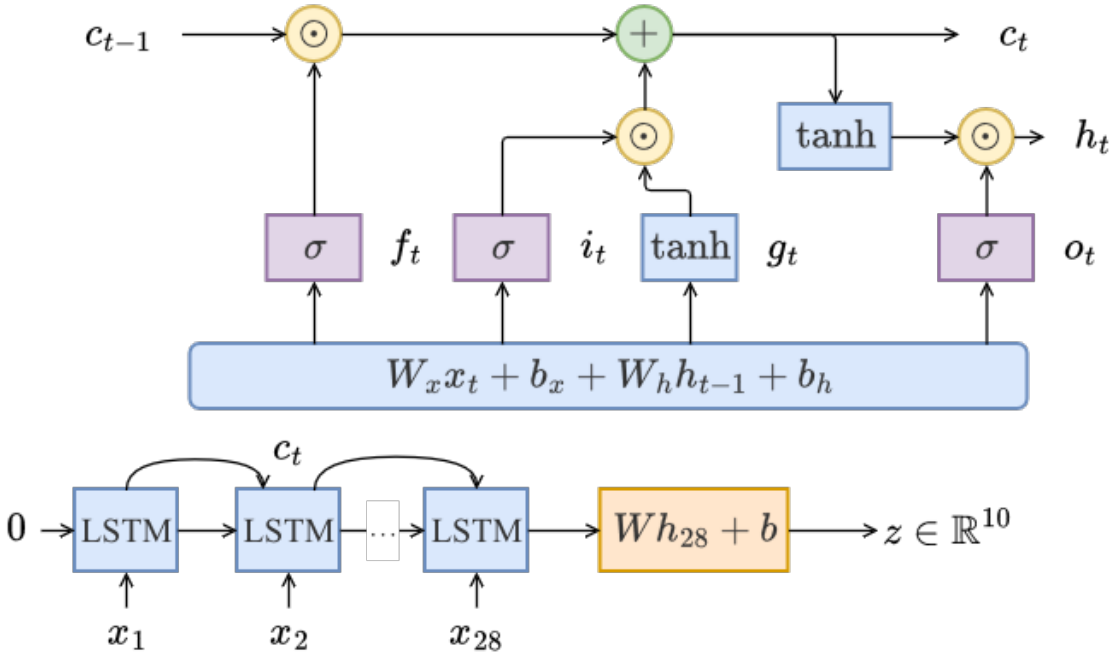


Figure 9: LSTM cell and row-sequence classifier.

2.2.6 Transformer

The Transformer uses the same row tokens as the recurrent models. Each token is projected to 128 dimensions and receives a sinusoidal positional encoding. Embedding dropout has probability 0.1. Two encoder layers each apply four-head self-attention and a feed-forward network of widths $128 \to 2048 \to 128$, with ReLU. Each sublayer uses a residual connection followed by layer normalization. Encoder dropout is 0.1 [10].

For a sequence representation $S \in \mathbb{R}^{H \times 128}$, one attention head computes

$$Q = SW_Q, \quad K = SW_K, \quad V = SW_V, \qquad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{32}}\right)V, \quad (1)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{128 \times 32}$ and the softmax acts across key positions. Projection biases are omitted from this notation. The four head outputs are concatenated and projected to 128 dimensions. The mean of the encoded row vectors feeds a linear classifier. Attention connects all row positions, while positional encoding supplies their order. Unlike the recurrent models, attention relates rows directly rather than passing information through every intervening row. Mean pooling then combines the encoded rows into one vector for classification.

2.3 Architecture comparison

The models use the same epoch budget but differ in representation, size, and regularization.

The group implements the model assembly, row-sequence processing, and GRU and LSTM cell updates. Affine layers, convolution, normalization, pooling, and the Transformer encoder use PyTorch modules [7].

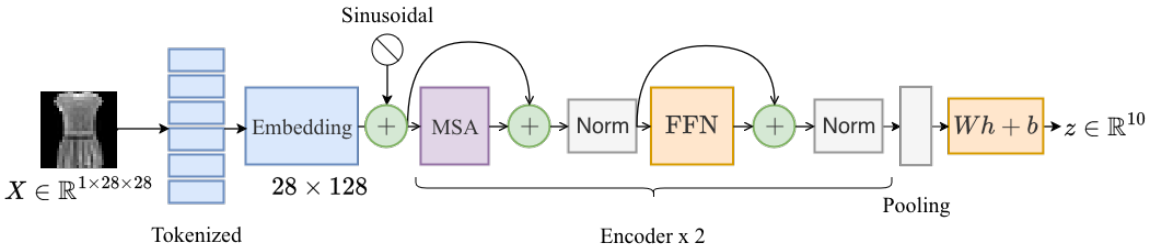


Figure 10: Transformer with row embeddings, positional encoding, two encoder layers, and mean pooling.

Table 2: Classifier dimensions and trainable parameters. MNIST and Fashion-MNIST share the same shapes.

Model	Feature dimension d	MNIST / Fashion-MNIST	CIFAR-10
Linear	784 / 3072	7,850	30,730
MLP	128	235,146	820,874
CNN	128	421,738	545,194
GRU	128	61,962	88,074
LSTM	128	82,186	117,002
Transformer	128	1,191,050	1,199,754

2.4 Loss and evaluation metrics

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote a labelled sample, where $y_i \in \{0, \ldots, K-1\}$ and $K = 10$. For an input x_i, the model produces a vector of logits $z_i = f_\theta(x_i) \in \mathbb{R}^K$. The corresponding class probabilities and predicted label are

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{j=0}^{K-1} \exp(z_{ij})}, \qquad \hat{y}_i = \arg\max_k z_{ik}. \tag{2}$$

Training objective. All models minimize the mean categorical cross-entropy,

$$\mathcal{L}_{\mathrm{CE}}(\theta; \mathcal{D}) = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i} = \frac{1}{N} \sum_{i=1}^{N} \left[\log \sum_{k=0}^{K-1} \exp(z_{ik}) - z_{i,y_i} \right]. \tag{3}$$

We compute cross-entropy directly from logits without applying softmax first. Each optimization step uses the mean loss of its minibatch. Cross-entropy penalizes a low probability for the true class, so two predictions with the same argmax can contribute different losses. Validation loss therefore considers confidence in addition to whether the predicted label is correct.

Classification performance. Define the confusion matrix C by

$$C_{ab} = \sum_{i=1}^{N} \mathbf{1}\{y_i = a,\ \hat{y}_i = b\}, \tag{4}$$

where rows correspond to true classes and columns to predicted classes. Accuracy is the proportion of correct predictions,

$$\text{Accuracy} = \frac{1}{N} \sum_{k=0}^{K-1} C_{kk}. \tag{5}$$

For class k, let $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_{a \neq k} C_{ak}$, and $\mathrm{FN}_k = \sum_{b \neq k} C_{kb}$. We report the unweighted mean of the class F1 scores,

$$F_{1,k} = \frac{2\mathrm{TP}_k}{2\mathrm{TP}_k + \mathrm{FP}_k + \mathrm{FN}_k}, \qquad \text{Macro-F1} = \frac{1}{K} \sum_{k=0}^{K-1} F_{1,k}. \tag{6}$$

Macro-F1 gives each class equal weight. We compute both metrics over the full evaluation split. Accuracy measures how often the predicted label is correct, while macro-F1 combines precision and recall for each class before averaging. Since Fashion-MNIST is balanced, differences between these metrics reflect the pattern of class errors rather than unequal numbers of test images.

Class-level analysis. Per-class recall measures the fraction of examples of a true class that the model identifies correctly,

$$R_k = \frac{C_{kk}}{\sum_b C_{kb}}. \tag{7}$$

We normalize each row as $\widetilde{C}_{ab} = C_{ab} / \sum_j C_{aj}$. The diagonal gives class recall. Off-diagonal values show the fraction of each true class assigned to an incorrect class.

Model size and computation. We count trainable parameters in the encoder and classification head, excluding buffers and optimizer state. Parameter count measures storage for model weights, not the operations needed for a prediction.

2.5 Training configuration

Table 3: Training settings for the reported runs.

Setting	Value
Optimizer	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Learning rate	10^{-3}, constant
Batch size	128
Epochs / seed	50 / 42
Validation fraction	0.1 of the official training split
Weight decay	0
Arithmetic	Float32, no mixed precision
Checkpoint rule	Minimum validation cross-entropy
MLP dropout	0.2 after each hidden activation
Transformer dropout	0.1 in embeddings and encoder layers
CNN normalization	Batch normalization after each convolution

All models use the preprocessing and augmentation in Section 1.5. CNN evaluation uses an NVIDIA GeForce RTX 4060 Ti with Python 3.11.11 and PyTorch 2.14.0+cu126. Other runs use macOS ARM and CUDA environments.

2.6 Comparison protocol

We compare Linear, MLP, CNN, GRU, LSTM, and Transformer on Fashion-MNIST. Within each dataset, all models share the split, preprocessing, augmentation, optimizer settings, epoch

budget, and checkpoint rule. This compares the models under one common training recipe. It does not give each architecture a separately tuned optimizer or schedule, so the ranking applies to this recipe rather than the best result each architecture could achieve.

For the detailed comparison with CNN, we choose the flattened-pixel baseline with the lower minimum validation loss among Linear and MLP. A tie favours Linear. This selects MLP. Both models are evaluated on the same official test images. Confusion matrices and class recall use all test predictions. For the qualitative comparison, we show the first two test images in each of three categories: CNN correct and MLP wrong, both wrong, and MLP correct and CNN wrong. We count the full test set separately to measure how often each outcome occurs.

3 Implementation results

3.1 Test performance and model size

CNN achieves 92.59% accuracy and a macro-F1 of 0.9261 on the official Fashion-MNIST test set. It exceeds Transformer accuracy by 2.16 percentage points while using 421,738 parameters compared with 1,191,050.

Table 4: Fashion-MNIST test results. Each model trains for 50 epochs with seed 42. We evaluate the checkpoint with the lowest validation loss.

Model	Accuracy (%)	Macro-F1	Parameters
Linear	77.05	0.7694	7,850
MLP	87.18	0.8711	235,146
CNN	92.59	0.9261	421,738
GRU	89.88	0.8990	61,962
LSTM	89.52	0.8952	82,186
Transformer	90.43	0.9040	1,191,050

Table 5: Selected checkpoint epoch and validation loss for each Fashion-MNIST model.

Model	Training epochs	Best epoch	Validation loss
Linear	50	14	0.6540
MLP	50	49	0.3413
CNN	50	42	0.1948
GRU	50	49	0.2676
LSTM	50	46	0.2754
Transformer	50	44	0.2630

MLP improves accuracy over Linear by 10.13 percentage points, while CNN adds another 5.41 points over MLP. Nonlinear models therefore improve substantially on the linear baseline in this comparison. The smaller gaps among CNN, the recurrent models, and Transformer make class-level results useful for seeing where their predictions differ.

3.2 Learning behavior

CNN validation loss reaches its minimum of 0.1948 at epoch 42. Table 5 shows that the selected epochs range from 14 for Linear to 49 for MLP and GRU. Training uses random augmentation and model-specific regularization, while validation disables random augmentation and dropout and uses running statistics for batch normalization. The loss curves therefore reflect different conditions in addition to the models' learning progress. Selecting epochs 49 for MLP and GRU places their best checkpoints close to the end of the budget, whereas Linear selects epoch 14. This does not establish that more epochs would improve MLP or GRU, but it shows that the models reach their lowest observed validation losses at different stages.

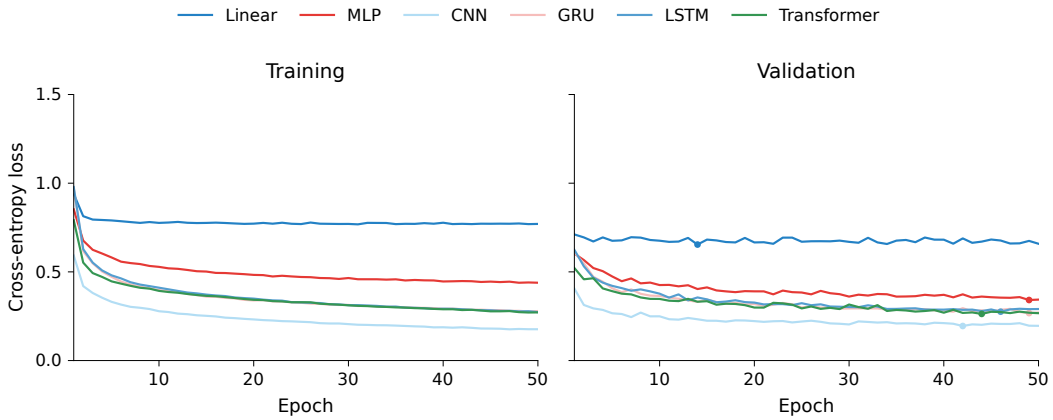


Figure 11: Fashion-MNIST training and validation cross-entropy for six models. Dots mark the selected checkpoints.

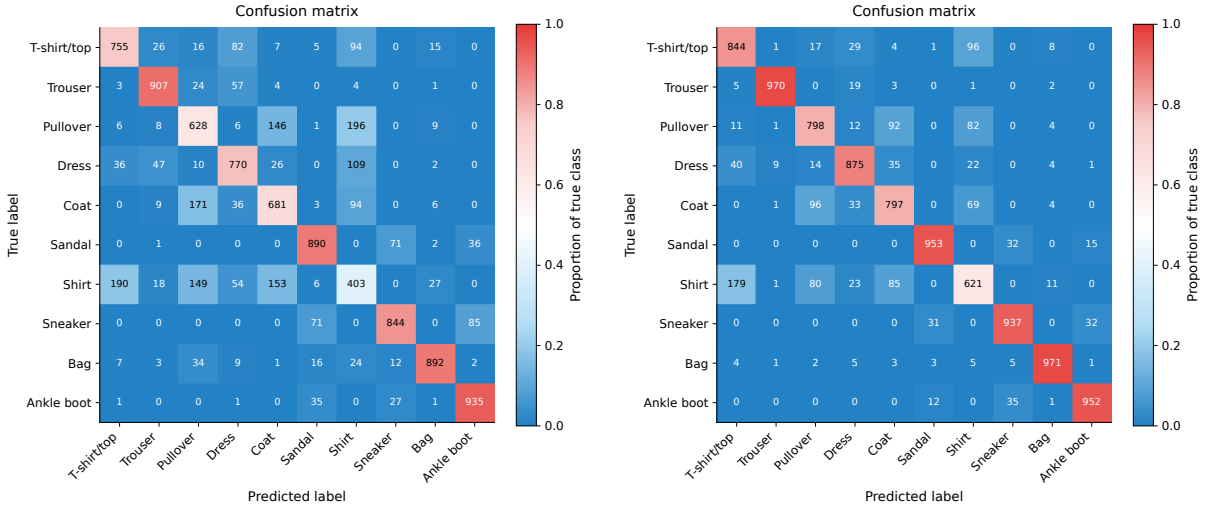
3.3 Per-class performance and error analysis

Confusions among T-shirt/top, Pullover, Coat, and Shirt account for 507 of CNN's 741 errors, or 68.4%. Here, both the true and predicted labels are in this group. CNN correctly identifies 79.5% of Shirt images, compared with 62.1% for MLP. It misclassifies 81 Shirt images as T-shirt/top and 61 Coat images as Pullover. The upper-body classes have similar average shapes in Figure 2, and they also account for most CNN errors. We have not tested which image details cause these errors.

Table 6: Fashion-MNIST test recall by class (%). Each class has 1,000 images.

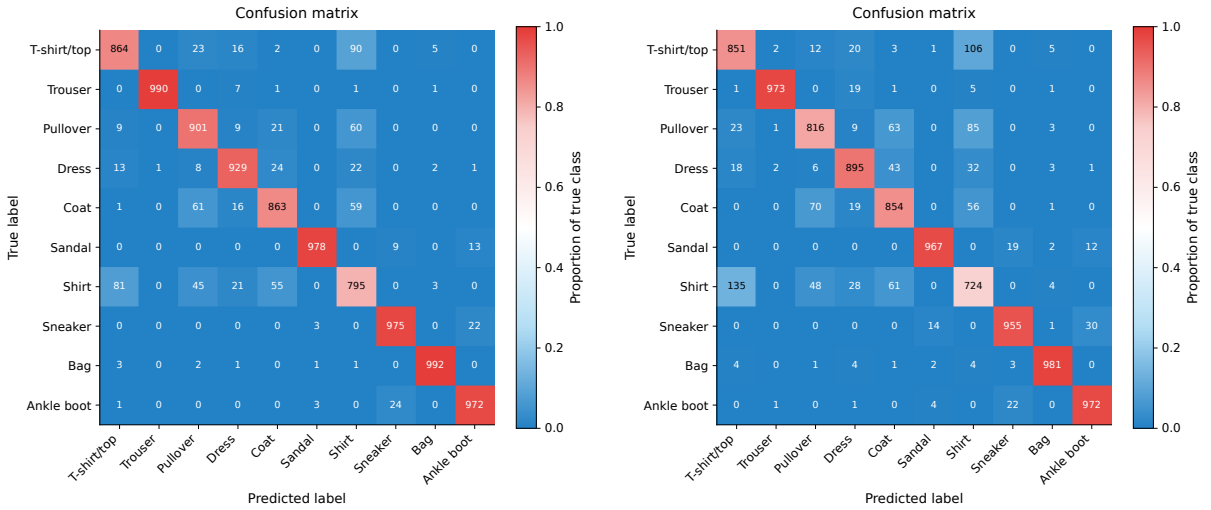
Class	Linear	MLP	CNN	GRU	LSTM	Transformer
T-shirt/top	75.5	84.4	86.4	85.1	83.0	81.8
Trouser	90.7	97.0	99.0	97.3	98.3	98.0
Pullover	62.8	79.8	90.1	81.6	81.6	84.0
Dress	77.0	87.5	92.9	89.5	88.7	93.2
Coat	68.1	79.7	86.3	85.4	84.6	85.6
Sandal	89.0	95.3	97.8	96.7	95.8	96.6
Shirt	40.3	62.1	79.5	72.4	70.9	72.3
Sneaker	84.4	93.7	97.5	95.5	98.7	97.2
Bag	89.2	97.1	99.2	98.1	98.2	98.7
Ankle boot	93.5	95.2	97.2	97.2	95.4	96.9

CNN's advantage is not uniform across classes. It reaches 99.2% recall for Bag and 99.0% for Trouser, while Shirt remains its weakest class at 79.5%. LSTM, on the other hand, has higher Sneaker recall than CNN at 98.7% versus 97.5%, and Transformer has slightly higher Dress recall at 93.2% versus 92.9%. Overall accuracy therefore hides some class-specific advantages of the lower-ranked models.



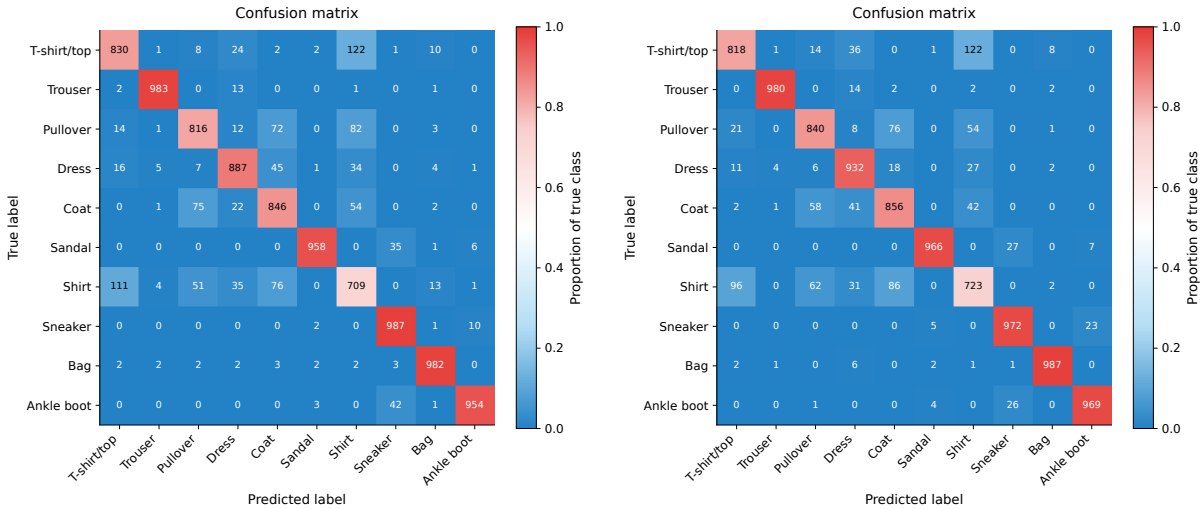
Linear MLP

Figure 12: Linear and MLP confusion matrices on the Fashion-MNIST test set. Rows show true classes and columns show predictions. Cell labels give counts, and color gives the fraction of each true class.



CNN GRU

Figure 13: CNN and GRU confusion matrices on the Fashion-MNIST test set. Rows show true classes and columns show predictions. Cell labels give counts, and color gives the fraction of each true class.



LSTM Transformer

Figure 14: LSTM and Transformer confusion matrices on the Fashion-MNIST test set. Rows show true classes and columns show predictions. Cell labels give counts, and color gives the fraction of each true class.

3.4 Predictions on shared images

CNN correctly classifies 775 images that MLP misses, while MLP correctly classifies 234 images that CNN misses. Both models misclassify 507 images and correctly classify 8,484. These counts distinguish the overall accuracy gain from a consistent improvement on every image. In particular, the 234 MLP-only successes identify cases worth inspecting even though CNN performs better on the full test set.

Figure 15 shows five models on the same six images. Each disagreement category has two examples, so the figure does not show how often each outcome occurs. LSTM is omitted because its per-image predictions are unavailable for this figure.

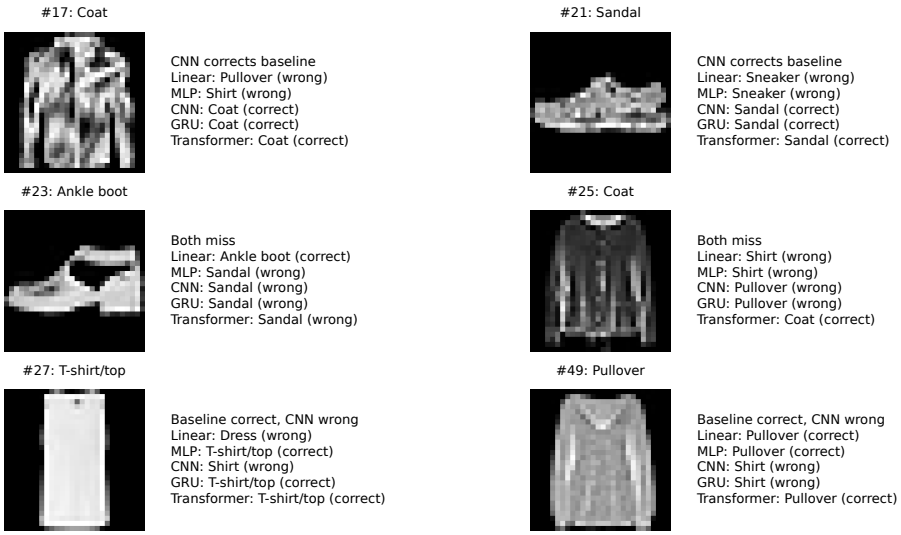


Figure 15: Fashion-MNIST predictions on the first two test images in each category: CNN correct and MLP wrong, both wrong, and MLP correct and CNN wrong. Indices are zero-based. Baseline denotes MLP.

The selected examples show several kinds of disagreement. On test image 25, which is a Coat, CNN and MLP predict Pullover and Shirt respectively, while Transformer predicts Coat correctly. On image 23, an Ankle boot, Linear is correct even though CNN and MLP both predict Sandal. These examples show why a shared error between the two main models should not automatically be treated as an unrecognizable image.

3.5 Limitations

We use one seed per model and do not run a systematic hyperparameter search or component ablation, so we cannot assess variation across training runs. Architecture, model size, and regularization change together, which prevents us from isolating the effect of each choice. Training and inference times also use different hardware or measurement conditions. Since inference timings lack warm-up and repeated trials, we do not compare model speed.

3.6 Discussion and conclusion

CNN achieves the highest Fashion-MNIST test accuracy at 92.59%, exceeding the Transformer by 2.16 percentage points while using 421,738 parameters compared with 1,191,050. The larger Transformer therefore offers no accuracy advantage in this experiment. On the other hand, GRU reaches 89.88% with just 61,962 parameters, making it a smaller alternative when model storage matters more than the additional 2.71 percentage points achieved by CNN. These results show why parameter count and accuracy should be considered together when choosing among the tested models.

One possible explanation for the Transformer result is the amount of training data relative to what the model must learn from scratch. CNN filters already connect nearby pixels and share weights across image positions, whereas our Transformer projects whole rows into tokens and learns relationships between them through attention. Vision Transformer research links weaker built-in spatial assumptions to greater data requirements [11]. Here, the Transformer learns from 54,000 training images without pretraining, so its larger capacity does not guarantee better generalization. We did not vary the training-set size, however, and cannot attribute the accuracy gap to data volume alone. Data-efficient Transformer training also shows that training strategy can reduce the need for external data [12].

The visual setting also helps explain why convolution is a suitable choice. Fashion-MNIST images contain isolated products on mostly dark backgrounds, with fewer distractions than full scenes containing clutter or several objects. Under these conditions, local edges, sleeves, and garment outlines offer useful cues for classification, while the benefit of connecting distant image regions through attention is less obvious. The task still has difficult distinctions. Confusions among upper-body garments account for 68.4% of CNN errors, so a simple background does not make Shirt, Coat, and Pullover easy to separate.

For this dataset and training setup, CNN is the strongest tested choice for accuracy, while GRU provides a smaller model with a moderate accuracy cost. The Transformer is not a better alternative under the current protocol, but this result does not rule out improvements from pretraining, different image tokens, or a training schedule designed for Transformers. MLP also correctly classifies 234 images that CNN misses, showing that the best overall model still has individual failures. A useful follow-up would vary training-set size for both CNN and Transformer under fixed settings and repeat the runs across seeds. This would test the data-size explanation directly rather than infer it from one accuracy comparison.

AI usage disclosure

The group states that it uses AI tools for project scaffolding, model checks, and guidance on report editing. Details are enclosed in `AI_USAGE.md`.

References

[1] Le Thanh Sach, *Course Project Handbook: Assignments 1–3, Deep Learning and Its Applications, CO3133, Semester 261*, HCMUT, revision 14 September 2026.

[2] Group Placeholder, *CO3133 Deep Learning course project repository*. https://github.com/vietanh1802/CO3133-Deep-Learning-261-Placeholder.

[3] Han Xiao, Kashif Rasul, and Roland Vollgraf, *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747, 2017. https://arxiv.org/abs/1708.07747.

[4] Zalando Research, *Fashion-MNIST dataset and distribution files*. https://github.com/zalandoresearch/fashion-mnist. Accessed 27 October 2026.

[5] PyTorch contributors, *torchvision.datasets.MNIST documentation*. Online documentation. Accessed 27 October 2026.

[6] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton, *CIFAR-10 and CIFAR-100 datasets*. https://cave.cs.toronto.edu/kriz/cifar.html. Accessed 27 October 2026.

[7] PyTorch contributors, *torch.nn documentation*, version 2.14. https://docs.pytorch.org/docs/2.14/nn.html. Accessed 28 October 2026.

[8] PyTorch contributors, *GRU documentation*, version 2.14. Online documentation. Accessed 28 October 2026.

[9] PyTorch contributors, *LSTM documentation*, version 2.14. Online documentation. Accessed 28 October 2026.

[10] PyTorch contributors, *TransformerEncoderLayer documentation*, version 2.14. Online documentation. Accessed 28 October 2026.

[11] Alexey Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR, 2021. https://arxiv.org/abs/2010.11929.

[12] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou, *Training data-efficient image transformers & distillation through attention*, ICML, 2021. https://arxiv.org/abs/2012.12877.